hashlock.

2024

Security Audit

U2U (Campaign)

November 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

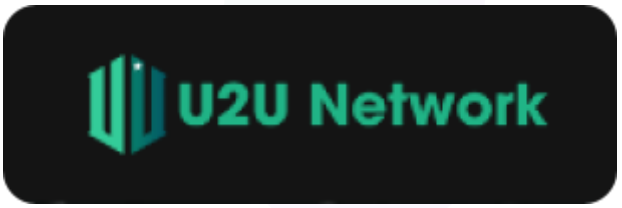
Executive Summary

The U2U team partnered with Hashlock to conduct a security audit of their IncentivePool.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

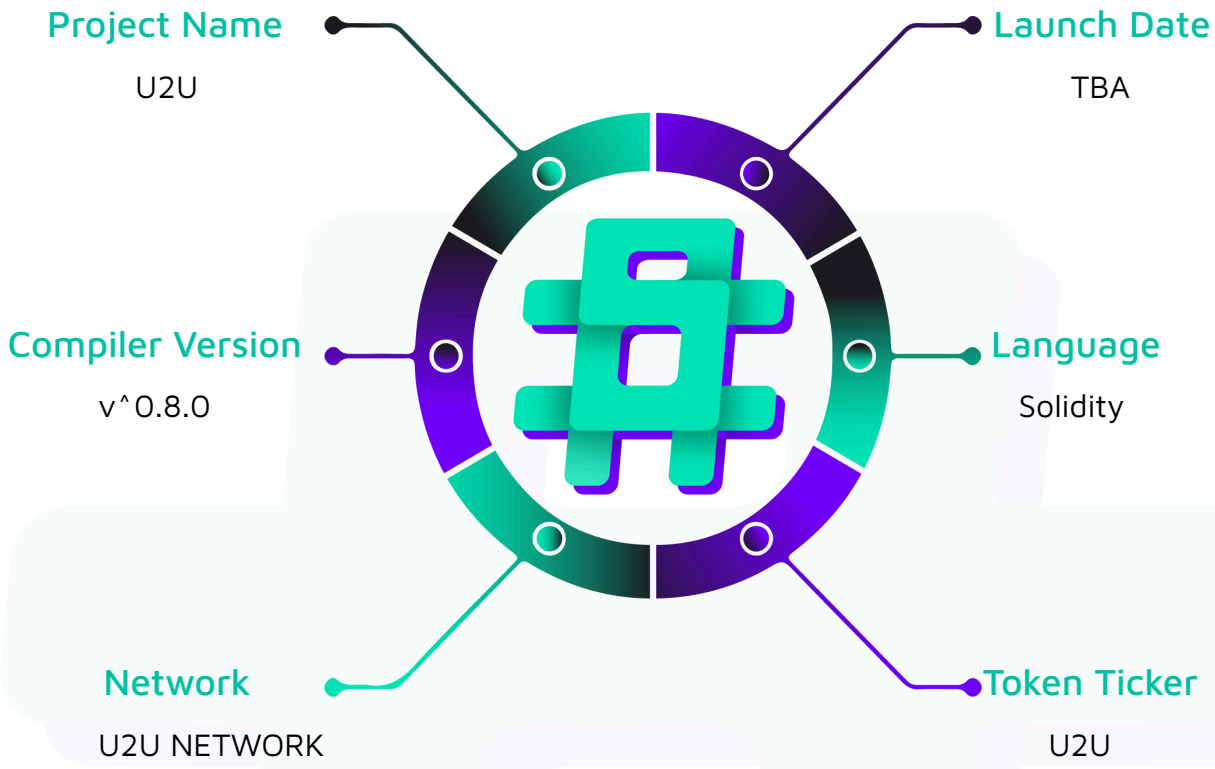
Project Context

The U2U Network is a decentralized blockchain platform designed to support scalable real-world applications, leveraging a Direct Acyclic Graph (DAG)-based architecture and EVM compatibility for fast, secure, and highly efficient operations. It aims to empower decentralized services, with innovations like DePIN (Decentralized Physical Infrastructure Networks) and modular subnet technology, allowing for high throughput (up to 72,000 TPS) and rapid transaction finality (650ms). U2U focuses on enabling decentralized privacy networks, IoT integration, decentralized storage, and digital identity solutions. This infrastructure aims to drive a decentralized, scalable future for Web3 applications.

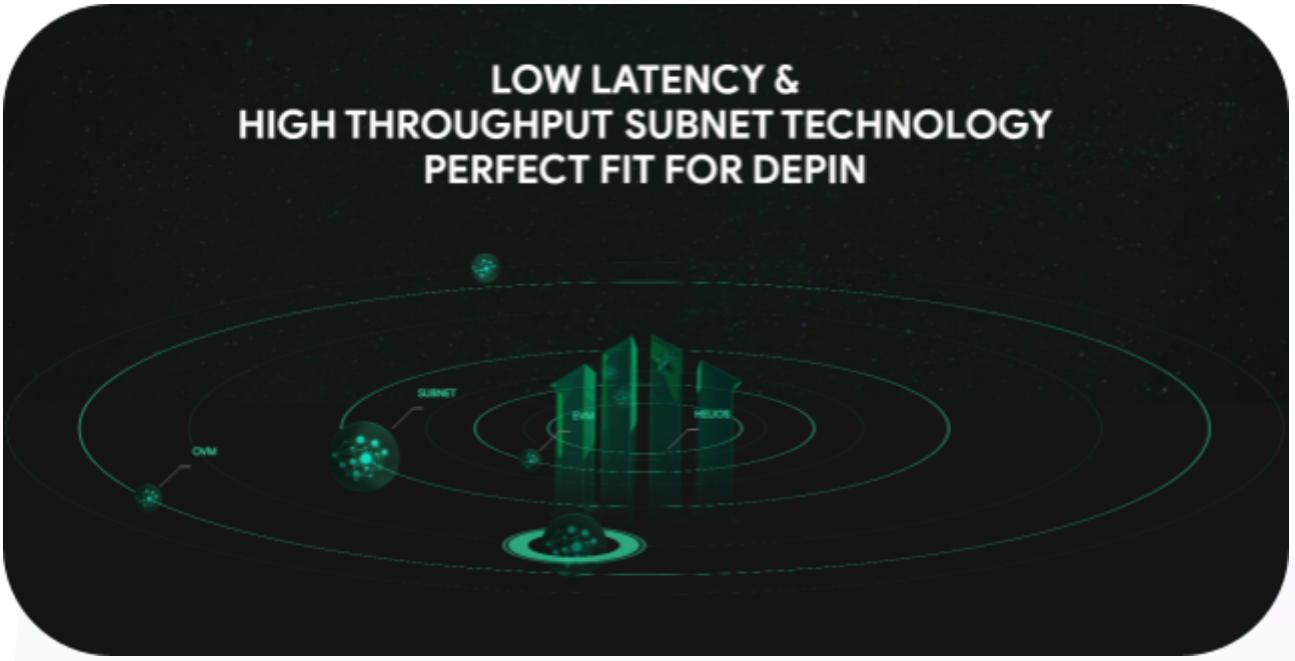
Project Name: U2U
Compiler Version: ^0.8.0
Website: www.u2u.xyz
Logo:



Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the U2U project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**U2U Protocol Smart Contracts**
Platform	**U2U Network / Solidity**
Audit Date	**November, 2024**
Contract 1	IncentivePool.sol
Contract 1 MD5 Hash	91307b9c1bd18cf821c38fe78933887c
GitHub Commit Hash	f84f1eebc037e4918076ccf5c372b7c3ac8b5269

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

3 Medium severity vulnerabilities

4 Low severity vulnerabilities

6 Gas Optimisations

3 QA Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
IncentivePool.sol - Allows users to: - Stake tokens within set limits and pool duration. - Harvest rewards based on staked amount and time. - Unstake tokens after pool ends. - Allows admins to: - Pause/unpause contract actions. - Toggle `ignoreSigner` for staking. - Emergency withdraw U2U rewards.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the U2U project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the U2U project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

High

[H-01] IncentivePool#onlyClaimableTime - Modifier is bypassable

Description

The `onlyClaimableTime` modifier is implemented in the `harvest` function. However, this is problematic as the `stake` function directly calls the `_harvest` function, bypassing this modifier.

Vulnerability Details

The `onlyClaimableTime` modifier checks if the `claimableTime` value has been reached.

```
   modifier onlyClaimableTime {
            require(claimableTime > 0 && block.timestamp >= claimableTime, "Harvest: unclaimable time");
        _;
    }
```

This modifier is only implemented in the `harvest` function.

```
   function harvest() external whenNotPaused nonReentrant onlyClaimableTime {
        _harvest();
    }

    function _harvest() internal {
        unchecked {
            address _user = msg.sender;
            uint256 _u2uRewards = _pendingRewards(_user);
            users_[_user].latestHarvest = block.timestamp;
            if (_u2uRewards > 0) {
                users_[_user].totalClaimed += _u2uRewards;
                // Handle send rewards
```

```
                TransferHelper.safeTransferNative(_user, _u2uRewards);
                emit Harvest(_user, _u2uRewards);
            }
        }
    }
```

Taking a look at the `stake` function below:

```
    function stake(
        uint256 _amount,
        uint256 _expiresAt,
        bytes memory _signature
    ) external whenNotPaused nonReentrant {
        require(block.timestamp < endTime, "Pool is ended");
        require(_amount >= MIN_STAKE_AMOUNT, "not enough minimum stake amount");
        if (!ignoreSigner) {
            require(!usedSignatures[_signature], "Stake: signature reused");
            usedSignatures[_signature] = true;
            address _signer = _verifyStake(_expiresAt, _signature);
            require(hasRole(POOL_SIGNER, _signer), "Stake: only signer");
            require(_expiresAt >= block.timestamp, "Stake: signature expired");
        }
        _stake(_amount);
    }

    function _stake(uint256 _amount) internal {
        unchecked {
            _harvest();
        // rest of the function
```

It is observed that this function directly makes a call to the `_harvest` function, bypassing the `onlyClaimableTime` modifier. This means that users can claim their rewards before `claimableTime` is reached by staking the minimum amount that is required.

This vulnerability also exists in the `unstake` function, however, since this function checks if the `endTime` has been reached it is not a realistic attack vector as long as `claimableTime` is less than the `endTime`.

Proof of Concept

An example of a test that simulates the vulnerability is below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import "../IncentivePool.sol";
import "../../libs/TransferHelper.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockERC20 is ERC20 {
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {}

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract IncentivePoolTest is Test {
    IncentivePool public incentivePool;
```

```
    MockERC20 public pUSDT;

    address user = address(0x1);
    address admin = address(0x2);

    uint256 constant INITIAL_PUSDT_SUPPLY = 1_000_000 * 1e6;
    uint256 constant START_TIME = 1_700_000_000;
    uint256 constant STAKE_AMOUNT = 1000 * 1e6;

    function setUp() public {
        // Deploy Mock pUSDT token and mint to user
        pUSDT = new MockERC20("Mock pUSDT", "pUSDT");
        pUSDT.mint(user, INITIAL_PUSDT_SUPPLY);

        // Deploy IncentivePool contract
        incentivePool = new IncentivePool(address(pUSDT), START_TIME);

        // Assign DEFAULT_ADMIN_ROLE to admin
        incentivePool.grantRole(incentivePool.DEFAULT_ADMIN_ROLE(), admin);

        // Set ignoreSigner to true
        vm.prank(admin);
        incentivePool.setIgnoreSigner(true);

        // Set claimableTime
        vm.prank(admin);
        incentivePool.setClaimableTime(START_TIME + 500);

        // Approvals
        vm.prank(user);
        pUSDT.approve(address(incentivePool), type(uint256).max);

        // Fund the IncentivePool contract
        vm.deal(address(incentivePool), 100000 ether);
```

```
}

function testStakeTwiceBeforeClaimableTimeRewardsDistributed() public {
    // Set claimableTime in the future
    uint256 futureClaimableTime = START_TIME + 1000;
    vm.prank(admin);
    incentivePool.setClaimableTime(futureClaimableTime);

    // Forward time to after start time but still before claimableTime
    vm.warp(START_TIME + 1);

    // Initial stake by user
    uint256 initialStakeAmount = 1000 * 1e6; // 1,000 pUSDT
    vm.prank(user);
    incentivePool.stake(initialStakeAmount, START_TIME + 1000, "");

    // Cache user's balance
    uint256 initialTotalClaimed = user.balance;

    // Move forward in time, but still before claimableTime
    vm.warp(START_TIME + 500);

    // Stake again with an additional amount
    uint256 additionalStakeAmount = 500 * 1e6; // 500 pUSDT
    vm.prank(user);
    incentivePool.stake(additionalStakeAmount, START_TIME + 1500, "");

    // Cache user's new balance
    uint256 totalClaimedAfterSecondStake = user.balance;

    // Verify rewards were distributed on the second stake
    assertGt(totalClaimedAfterSecondStake, initialTotalClaimed);

    // Print user's claimed rewards for verification
```

```
        console.log("User's rewards after first stake:", initialTotalClaimed);
        console.log("User's rewards after second stake:", totalClaimedAfterSecondStake);
    }
}
```

Impact

Users can claim rewards even if `claimableTime` is less than the `block.timestamp`.

Recommendation

Change the `_stake` and `_unstake` functions so that these functions keep track of how many rewards are owed to the user at that time instead of calling the `_harvest` function. With this change, the `_harvest` function should transfer users the owed rewards and the pending rewards. Implement correct calculations to prevent double claiming of rewards.
In short, implement a snapshot mechanic that keeps track of rewards that are owed to users.

Status

Resolved

[H-02] IncentivePool#_pendingRewards - Users keep accumulating rewards after the pool ends

Description

The `_pendingRewards` function calculates a user's rewards based on the amount of time that has passed since the last `_harvest` call, the number of tokens staked, and the rewards rate per second. However, this function does not take into account when the staking pool ends, causing users to accumulate more rewards after the pool ends.

Vulnerability Details

The `_pendingRewards` function calculates a user's rewards.

```
    function _pendingRewards(address _user) internal view returns (uint256) {
        unchecked {
```

```
            (uint256 totalStaked, uint256 latestHarvest, ) = _getUserInfo(
                _user
            );
            if (block.timestamp < startTime) return 0;
            if (latestHarvest < startTime) {
                latestHarvest = startTime;
            }
            if (totalStaked == 0) return 0;
            uint256 timeRewards = block.timestamp - latestHarvest;
            return timeRewards.mul(totalStaked).mul(_rewardsRatePerSecond());
        }
    }
```

As observed in the highlighted part of the function, no checks were done on when the pool ended. This means that users will keep accumulating rewards after the pool ends.

This will cause discrepancies in the expected reward that will be paid to a user. Think of the scenario where 5 users have staked 100e6 tokens for the full pool duration. A user's reward would be: `100e6 * 90 days * 857338 = 666666028800000000000`. The protocol would be depositing approximately this amount times 5 to the contract in order to pay for users' rewards. However, since the rewards will be accumulating after the pool ends, if some users do not claim their rewards right when the pool ends, there might be insufficient U2U in the contract to pay for all rewards.

Another problematic scenario is when any user does not claim their rewards when the pool ends, rewards will keep accumulating and for example, when this user claims 270 days after they have staked, they would be eligible for approximately 1999e18 U2U, there is a chance the contract will not have enough tokens to pay for their rewards, reverting any `unstake` or `harvest` call and causing not only their rewards but also their staked tokens to be stuck in the contract until enough U2U has been funded.

Proof of Concept

An example of a test that simulates the vulnerability is below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import "../IncentivePool.sol";
import "../../libs/TransferHelper.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

// Mock ERC20 token to represent pUSDT
contract MockERC20 is ERC20 {
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {}

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract IncentivePoolTest is Test {
    IncentivePool public incentivePool;
    MockERC20 public pUSDT;

    address user = address(0x1);
    address admin = address(0x2);
    address user2 = address(0x3);
```

```
    uint256 constant INITIAL_PUSDT_SUPPLY = 1_000_000 * 1e6;
    uint256 constant START_TIME = 1730000000;
    uint256 constant END_TIME = START_TIME + 90 days;
    uint256 constant STAKE_AMOUNT = 1000 * 1e6;
    uint256 constant MIN_STAKE_AMOUNT = 10 * 1e6;
    uint256 constant MAX_USDT_POOL_CAP = 1_500_000 * 1e6;
    uint256 constant MAX_STAKE_AMOUNT = 10000 * 1e6;

    function setUp() public {
        // Deploy Mock pUSDT token and mint to users
        pUSDT = new MockERC20("Mock pUSDT", "pUSDT");
        pUSDT.mint(user, 10e6);
        pUSDT.mint(user2, 20e6);

        // Deploy IncentivePool contract
        incentivePool = new IncentivePool(address(pUSDT), START_TIME);

        // Assign DEFAULT_ADMIN_ROLE to admin
        incentivePool.grantRole(incentivePool.DEFAULT_ADMIN_ROLE(), admin);

        // Set ignoreSigner to true to bypass signature checks for testing
        vm.prank(admin);
        incentivePool.setIgnoreSigner(true);

        // Set claimableTime to allow harvesting
        vm.prank(admin);
        incentivePool.setClaimableTime(START_TIME + 1);

         // Approve the maximum allowance for IncentivePool to spend pUSDT on behalf of
the users
        vm.prank(user);
        pUSDT.approve(address(incentivePool), type(uint256).max);
        vm.prank(user2);
        pUSDT.approve(address(incentivePool), type(uint256).max);
```

```
        // Fund the IncentivePool contract with U2U (native asset) for rewards
        vm.deal(address(incentivePool), 10000 ether);
    }

    function testUserKeepsAccumulatingRewards() public {
        // Forward time to after start time
        vm.warp(START_TIME);

        // Stake the amount
        vm.prank(user);
        incentivePool.stake(10e6, END_TIME, "");
        vm.prank(user2);
        incentivePool.stake(10e6, END_TIME, "");

        // Forward time to when pool ends and user unstakes
        vm.warp(END_TIME + 1);
        vm.prank(user);
        incentivePool.unstake();

        // Forward time 10000000 seconds and user2 unstakes
        vm.warp(END_TIME + 10000001);
        vm.prank(user2);
        incentivePool.unstake();

        //Assertion and logs
        assertGt(user2.balance, user.balance);
        console.log("user1 bal:", user.balance);
        console.log("user2 bal:", user2.balance);
    }
```

Impact

Users will accumulate more rewards than it is expected. This will cause the user's rewards and staked tokens to be stuck in the contract.

Recommendation

Change the reward calculation logic to consider when the pool has ended. An example is shown below.

```
function _pendingRewards(address _user) internal view returns (uint256) {
    unchecked {
        (uint256 totalStaked, uint256 latestHarvest, ) = _getUserInfo(_user);

        if (block.timestamp < startTime) return 0;
        if (latestHarvest < startTime) {
            latestHarvest = startTime;
        }

        if (totalStaked == 0) return 0;

            uint256 endCalculationTime = block.timestamp < endTime ? block.timestamp : endTime;

        uint256 timeRewards = endCalculationTime - latestHarvest;
        return timeRewards.mul(totalStaked).mul(_rewardsRatePerSecond());
    }
}
```

Status

Resolved

Medium

[M-01] IncentivePool#constructor - No checks done to validate startTime will cause contract functionality failure

Description

In the `constructor`, `startTime` is set. No checks are done to ensure that the inputted `_startTime` is valid. Accidental inputs will break contract functionality.

Vulnerability Details

The `constructor` allows the deployer to set the `startTime`.

```
constructor(
    address _pUSDT,
    uint256 _startTime
) EIP712("IncentivePool", "1") {
    pUSDT = _pUSDT;
    startTime = _startTime;
    endTime = _startTime + MAX_STAKING_DAYS;
    _setupRole(DEFAULT_ADMIN_ROLE, msg.sender);
}
```

As observed, `startTime` is set to the inputted value, and the `endTime` is set to `_startTime + 90 days`. There are no checks done to validate the inputted `_startTime` value to be greater than or equal to `block.timestamp`. In the scenario where `startTime` is set to a value that is less than the `block.timestamp + 90 days`, it will be impossible for users to stake due to the requirement highlighted below.

```
function stake(
    uint256 _amount,
    uint256 _expiresAt,
    bytes memory _signature
) external whenNotPaused nonReentrant {
    require(block.timestamp < endTime, "Pool is ended");
```

```
        // rest of the function
```

Proof of Concept

An example of a test that simulates the vulnerability is below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import "../IncentivePool.sol";
import "../../libs/TransferHelper.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

// Mock ERC20 token to represent pUSDT
contract MockERC20 is ERC20 {
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {}

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract IncentivePoolTest is Test {
    IncentivePool public incentivePool;
    MockERC20 public pUSDT;

    address user = address(0x1);
    address admin = address(0x2);
```

```
    uint256 constant INITIAL_PUSDT_SUPPLY = 1_000_000 * 1e6;
    uint256 constant START_TIME = 1_700_000_000;
    uint256 constant END_TIME = START_TIME + 90 days;
    uint256 constant STAKE_AMOUNT = 1000 * 1e6;

    function setUp() public {
        // Deploy Mock pUSDT token and mint to user
        pUSDT = new MockERC20("Mock pUSDT", "pUSDT");
        pUSDT.mint(user, INITIAL_PUSDT_SUPPLY);

        // Deploy IncentivePool contract
        incentivePool = new IncentivePool(address(pUSDT), START_TIME);

        // Assign DEFAULT_ADMIN_ROLE to admin
        incentivePool.grantRole(incentivePool.DEFAULT_ADMIN_ROLE(), admin);

        // Set ignoreSigner to true to bypass signature checks for testing
        vm.prank(admin);
        incentivePool.setIgnoreSigner(true);

        // Set claimableTime to allow harvesting
        vm.prank(admin);
        incentivePool.setClaimableTime(START_TIME + 500);

         // Approve the maximum allowance for IncentivePool to spend pUSDT on behalf of
the user
        vm.prank(user);
        pUSDT.approve(address(incentivePool), type(uint256).max);

        // Fund the IncentivePool contract with U2U (native asset) for rewards
        vm.deal(address(incentivePool), 100000 ether);
    }

    function testStakeInvalidStartTime() public {
```

```
        // Forward time to a realistic block.timestamp
        vm.warp(1730691200);

        // Stake the amount and expect revert
        vm.prank(user);
        vm.expectRevert("Pool is ended");
        incentivePool.stake(STAKE_AMOUNT, block.timestamp + 1000, "");
    }
```

Impact

It will be impossible for users to stake in the protocol. This vulnerability breaks contract functionality.

Recommendation

Implement checks to validate the `startTime`. An example is shown below.

```
constructor(
    address _pUSDT,
    uint256 _startTime
) EIP712("IncentivePool", "1") {
    // rest of the function
    if (_startTime < block.timestamp) {
        revert;
    }
    // rest of the function
```

Status

Resolved

[M-02] IncentivePool#constructor - claimableTime is not set in the constructor

Description

The `claimableTime` sets a time when claiming user rewards becomes possible. However, this value is not set in the constructor. By default, this value will be 0 which will make it

impossible to claim rewards with `harvest` function due to the `onlyClaimableTime` modifier. This also creates a centralization risk.

Vulnerability Details

The `onlyClaimableTime` modifier blocks users from claiming their rewards with `harvest` function if the `claimableTime` is 0 or less than `block.timestamp`.

```
   modifier onlyClaimableTime {
            require(claimableTime > 0 && block.timestamp >= claimableTime, "Harvest: unclaimable time");
        _;
    }
    function harvest() external whenNotPaused nonReentrant onlyClaimableTime {
        _harvest();
    }
```

Taking a look at the `constructor` below:

```
   constructor(
        address _pUSDT,
        uint256 _startTime
    ) EIP712("IncentivePool", "1") {
        pUSDT = _pUSDT;
        startTime = _startTime;
        endTime = _startTime + MAX_STAKING_DAYS;
        _setupRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
```

It is observed that the `claimableTime` is not set, meaning that this value will be 0 and any calls to the `harvest` function will revert. The only way for the admin to change this value is by calling the `setClaimableTime` function shown below.

```
   function setClaimableTime (uint256 _time) external onlyMasterAdmin {
        require(_time >= block.timestamp, "invalid time");
        claimableTime = _time;
        emit UpdateClaimableTime(_time);
```

```
    }
```

Not having this value set in the constructor can cause the admin to never change this value due to a mistake or to act maliciously, making all `harvest` calls revert. This creates a centralization risk where the admin can avoid ever setting the `claimableTime` value.

Proof of Concept

An example of a test that simulates the vulnerability is below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import "../IncentivePool.sol";
import "../../libs/TransferHelper.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

// Mock ERC20 token to represent pUSDT
contract MockERC20 is ERC20 {
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {}

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract IncentivePoolTest is Test {
    IncentivePool public incentivePool;
    MockERC20 public pUSDT;
```

```
    address user = address(0x1);
    address admin = address(0x2);

    uint256 constant INITIAL_PUSDT_SUPPLY = 1_000_000 * 1e6;
    uint256 constant START_TIME = 1_700_000_000;
    uint256 constant END_TIME = START_TIME + 90 days;
    uint256 constant STAKE_AMOUNT = 1000 * 1e6;

    function setUp() public {
        // Deploy Mock pUSDT token and mint to user
        pUSDT = new MockERC20("Mock pUSDT", "pUSDT");
        pUSDT.mint(user, INITIAL_PUSDT_SUPPLY);

        // Deploy IncentivePool contract
        incentivePool = new IncentivePool(address(pUSDT), START_TIME);

        // Assign DEFAULT_ADMIN_ROLE to admin
        incentivePool.grantRole(incentivePool.DEFAULT_ADMIN_ROLE(), admin);

        // Set ignoreSigner to true to bypass signature checks for testing
        vm.prank(admin);
        incentivePool.setIgnoreSigner(true);

        // Set claimableTime to allow harvesting Commented out for test
        //vm.prank(admin);
        //incentivePool.setClaimableTime(START_TIME + 500);

         // Approve the maximum allowance for IncentivePool to spend pUSDT on behalf of
the user
        vm.prank(user);
        pUSDT.approve(address(incentivePool), type(uint256).max);

        // Fund the IncentivePool contract with U2U (native asset) for rewards
        vm.deal(address(incentivePool), 100000 ether);
```

```
    }

    function testStakeAndHarvest() public {
        // Forward time to after start time
        vm.warp(START_TIME + 1);

        // Stake the amount
        vm.prank(user);
        incentivePool.stake(STAKE_AMOUNT, START_TIME + 1000, "");

        // Check user's staked amount
        (uint256 userTotalStaked, , ) = incentivePool.getUserInfo(user);
        assertEq(userTotalStaked, STAKE_AMOUNT);

        // Check totalPoolStaked updated correctly
        assertEq(incentivePool.totalPoolStaked(), STAKE_AMOUNT);

        //Advance time and expect harvest to fail
        vm.warp(END_TIME + 1001);
        vm.prank(user);
        vm.expectRevert("Harvest: unclaimable time");
        incentivePool.harvest();

        assertEq(user.balance, 0);
    }
```

Impact

The `harvest` function will always revert blocking users from claiming their rewards.

Recommendation

Set the `claimableTime` value as the constructor. An example is shown below.

```
    constructor(
        address _pUSDT,
```

```
        uint256 _startTime
    ) EIP712("IncentivePool", "1") {
        pUSDT = _pUSDT;
        startTime = _startTime;
        endTime = _startTime + MAX_STAKING_DAYS;
        claimableTime = _startTime + 10 days;
        _setupRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
```

Status

Resolved

[M-03] IncentivePool#emergencyWithdrawU2U - Admin can withdraw rewards that are owed to users

Description

The `emergencyWithdrawU2U` function allows the admin to withdraw funds from the contract. This is assumed to be used in scenarios where an exploit creates an emergency for the admin to save the funds from the contract. However, this creates a significant centralization risk as the function does not account for the amount of rewards that are owed to staking users, allowing the admin to withdraw all funds.

Vulnerability Details

The `emergencyWithdrawU2U` function allows the admin to withdraw funds from the contract.

```
    function emergencyWithdrawU2U(
        address _to,
        uint256 _amount
    ) external onlyMasterAdmin {
        TransferHelper.safeTransferNative(_to, _amount);
    }
```

As observed in this function, there are no checks implemented and the admin can withdraw any amount they want.

The centralization risk this causes is when users have staked in the protocol and expect rewards, the admin can also withdraw the amounts that are owed to these stakers. Stakers will not be able to withdraw their rewards due to the contract not having enough balance.

Proof of Concept

An example of a test that simulates the vulnerability is below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import "../IncentivePool.sol";
import "../../libs/TransferHelper.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

// Mock ERC20 token to represent pUSDT
contract MockERC20 is ERC20 {
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {}

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract IncentivePoolTest is Test {
    IncentivePool public incentivePool;
    MockERC20 public pUSDT;
```

```
    address user = address(0x1);
    address admin = address(0x2);
    address user2 = address(0x3);

    uint256 constant INITIAL_PUSDT_SUPPLY = 1_000_000 * 1e6;
    uint256 constant START_TIME = 1730000000;
    uint256 constant END_TIME = START_TIME + 90 days;
    uint256 constant STAKE_AMOUNT = 1000 * 1e6;
    uint256 constant MIN_STAKE_AMOUNT = 10 * 1e6;
    uint256 constant MAX_USDT_POOL_CAP = 1_500_000 * 1e6;
    uint256 constant MAX_STAKE_AMOUNT = 10000 * 1e6;

    function setUp() public {
        // Deploy Mock pUSDT token and mint to users
        pUSDT = new MockERC20("Mock pUSDT", "pUSDT");
        pUSDT.mint(user, INITIAL_PUSDT_SUPPLY);
        pUSDT.mint(user2, INITIAL_PUSDT_SUPPLY);

        // Deploy IncentivePool contract
        incentivePool = new IncentivePool(address(pUSDT), START_TIME);

        // Assign DEFAULT_ADMIN_ROLE to admin
        incentivePool.grantRole(incentivePool.DEFAULT_ADMIN_ROLE(), admin);

        // Set ignoreSigner to true to bypass signature checks for testing
        vm.prank(admin);
        incentivePool.setIgnoreSigner(true);

        // Set claimableTime to allow harvesting
        vm.prank(admin);
        incentivePool.setClaimableTime(START_TIME + 500);

         // Approve the maximum allowance for IncentivePool to spend pUSDT on behalf of
the users
```

```
        vm.prank(user);
        pUSDT.approve(address(incentivePool), type(uint256).max);
        vm.prank(user2);
        pUSDT.approve(address(incentivePool), type(uint256).max);

        // Fund the IncentivePool contract with U2U (native asset) for rewards
        vm.deal(address(incentivePool), 10000 ether);
    }

    function testAdminCanStealRewards() public {
        // Forward time to after start time
        vm.warp(START_TIME + 1);

        // Stake the amount
        vm.prank(user);
        incentivePool.stake(STAKE_AMOUNT, END_TIME, "");
        vm.prank(user2);
        incentivePool.stake(STAKE_AMOUNT, END_TIME, "");

        // Advance time
        vm.warp(END_TIME + 10);

        // User2 harvest rewards
        vm.prank(user2);
        incentivePool.harvest();

        // Admin withdraws funds
        vm.prank(admin);
        incentivePool.emergencyWithdrawU2U(admin, address(incentivePool).balance);

        // Expect user's harvest to revert
        vm.prank(user);
        vm.expectRevert();
        incentivePool.harvest();
```

```
        // Assertions
        assertGt(user2.balance, user.balance);
        assertEq(0, user.balance);
    }
```

Impact

Users will be unable to withdraw their rewards

Recommendation

Keep track of the total amount of rewards that will be owed to the users at the `endTime`. Reduce this amount anytime a `_harvest` call happens. In the `emergencyWithdrawU2U` function, do not allow the admin to withdraw these amounts that are owed to users. An example of this is shown below.

```
    function emergencyWithdrawU2U(
        address _to,
        uint256 _amount
    ) external onlyMasterAdmin {
        if(_amount > address(this).balance - totalAmountOwedToUsers) revert;
        TransferHelper.safeTransferNative(_to, _amount);
    }
```

Status

Resolved

Low

[L-01] IncentivePool - Outdated version of Solidity

Description

The project uses `pragma solidity ^0.8.0`. 0.8.0 is an outdated version of Solidity

Recommendation

Use the latest version of Solidity. Be sure to check the known issues with the Solidity version chosen.

Status

Acknowledged

[L-02] IncentivePool#constructor - Missing check for address(0)

Description

Checking for `address(0)` in the constructor will prevent potential mistakes that would require redeployment.

Recommendation

Check for `address(0)` in the constructor. An example is shown below.

```
constructor(
    address _pUSDT,
    uint256 _startTime
) EIP712("IncentivePool", "1") {
    if (_pUSDT == address(0)) {
        revert;
    }
    // rest of the function
```

Status

Resolved

[L-03] IncentivePool - Avoid the use of floating pragma

Description

A "floating pragma" in Solidity refers to the practice of using a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This issue arises when pragma statements like `pragma solidity ^0.8.0;` are used without a specific version number, allowing the contract to be compiled with the latest available compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known issues for the compiler version that is chosen.

Status

Acknowledged

[L-04] IncentivePool - Centralization risk for privileged roles

Description

Privileged functions need the privileged role to be trusted not to perform malicious updates. This may also cause a single point failure. Functions with the centralization risk are shown below.

```
    function setIgnoreSigner(bool _state) external onlyMasterAdmin {

    function pause() external onlyMasterAdmin whenNotPaused {

    function unpause() external onlyMasterAdmin whenPaused {
```

```
    function emergencyWithdrawU2U(
        address _to,
        uint256 _amount
    ) external onlyMasterAdmin {
```

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders.

Status

Acknowledged

Gas

[G-01] IncentivePool#setIgnoreSigner, setClaimableTime - Prevent setting the state variable with the same value

Description

Not only is wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario.

Recommendation

Update the function to prevent setting the state variable with the same value. An example is shown below.

```
    function setIgnoreSigner(bool _state) external onlyMasterAdmin {
        if (_state == ignoreSigner) revert;
        ignoreSigner = _state;
        emit UpdateIgnoreSignerState(_state);
```

```
    }
    function setClaimableTime (uint256 _time) external onlyMasterAdmin {
        require(_time >= block.timestamp, "invalid claimable time");
        if(_time == claimableTime) revert;
        claimableTime = _time;
        emit UpdateClaimableTime(_time);
    }
```

Status

Resolved

[G-02] IncentivePool - It is unnecessary to use safeMath in solidity ^0.8.0

Description

Version 0.8.0 introduces internal overflow checks, so using `SafeMath` is redundant and adds unnecessary gas costs.

Recommendation

Discontinue using `SafeMath`.

Status

Acknowledged

[G-03] IncentivePool#_getUserInfo - Multiple accesses of a mapping should use a local variable cache

Description

Caching a mapping's value in a local storage or `calldata` variable when the value is accessed multiple times, saves ~42 gas per access due to not having to recalculate the key's keccak256 hash (`Gkeccak256` - 30 gas), and that calculation's associated stack operations.

Recommendation

Cache the mapping's value in the function. An example is shown below:

```
function _getUserInfo(
    address _user
)
    internal
    view
    returns (
        uint256 totalStaked,
        uint256 latestHarvest,
        uint256 totalClaimed
    )
{
    UserInfo memory user = users_[_user];
    totalStaked = user.totalStaked;
    latestHarvest = user.latestHarvest;
    totalClaimed = user.totalClaimed;
}
```

Status

Resolved

[G-04] IncentivePool - Missing constant modifier for state variables

Description

State variables that are never reassigned after their initial assignment should typically be declared with the `constant` modifier. This ensures that these variables remain unmodified and communicate their unchanging nature. Using the `constant` modifier also offers potential gas savings, as accessing these variables does not require any storage operations.

Recommendation

Declare state variables that are not reassigned after initialization with the 'constant' modifier. An example is shown below.

```
uint256 public constant MAX_USDT_POOL_CAP = 1_500_000 * 1e6;
```

```
uint256 public constant MAX_U2U_REWARDS = 10_000_000 * 1e18;
uint256 public constant MIN_STAKE_AMOUNT = 10 * 1e6;
uint256 public constant MAX_STAKE_AMOUNT = 10000 * 1e6;
uint256 public constant MAX_STAKING_DAYS = 90 days;
```

Status

Resolved

[G-05] IncentivePool - State variables only set in the constructor should be declared immutable

Description

State variables that are only set in the constructor should be declared `immutable`. This improves the run time and deployment gas costs.

Recommendation

Declare state variables that are only set in the constructor with the `immutable` modifier. An example is shown below.

```
address public immutable pUSDT;
uint256 public immutable startTime;
uint256 public immutable endTime;
```

Status

Resolved

[G-06] IncentivePool#verifyStake - public functions not called by the contract should be declared external instead

Description

When a function is not used internally within a contract and is only intended for external calls, it should be labelled as external rather than public. Using the public unnecessarily can introduce potential vulnerabilities and also make the contract consume more gas than required.

Recommendation

When a function is not used internally within a contract and is only intended for external calls, it should be labeled as `external` rather than `public`. Using `public` unnecessarily can introduce potential vulnerabilities and also make the contract consume more gas than required

```
function verifyStake(
    uint256 _expiresAt,
    bytes memory _signature
) external view returns (address) {
```

Status

Resolved

QA

[Q-01] IncentivePool#_pendingRewards - Constants in comparisons should appear on the left side

Description

Doing so will prevent typo bugs.

Recommendation

Adjust the function as shown below.

```
  function _pendingRewards(address _user) internal view returns (uint256) {
      unchecked {
          (uint256 totalStaked, uint256 latestHarvest, ) = _getUserInfo(
              _user
          );
          if (0 == totalStaked) return 0;
          // rest of the function
```

Status

Acknowledged

[Q-02] IncentivePool - Function ordering does not follow the Solidity style guide

Description

According to the Solidity style guide, functions should be laid out in the following order: `constructor()`, `receive()`, `fallback()`, `external`, `public`, `internal`, and `private`.

Recommendation

Follow the official Solidity guidelines.

Status

Acknowledged

[Q-03] IncentivePool - Dependence on external protocols

Description

External protocols should be monitored as such dependencies may introduce vulnerabilities if a vulnerability is found or introduced in the external protocol. Contract relies heavily on OpenZeppelin libraries.

Recommendation

Regularly monitor and review the external protocols your Solidity contracts depend on for any updates or identified vulnerabilities. Consider implementing fallback mechanisms or contingency plans in your contracts to handle potential failures or compromises in these external protocols.

Status

Acknowledged

Centralisation

The U2U project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the U2U project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au